

Pré-Projeto — Auditor Automático de E-commerce

Minimal · Sistema interno de verificação técnica da loja
Documento de briefing — nível estratégico/conceitual

1. Resumo executivo

Quando o faturamento da loja cai, uma das primeiras hipóteses a descartar é que algo quebrou ou degradou tecnicamente no e-commerce: uma página fora do ar, um botão de compra que parou de responder, o checkout travando em alguma etapa, uma imagem que não carrega, um erro de JavaScript, a performance despencando, um link quebrado. Hoje, validar isso significa abrir página por página e testar fluxo por fluxo manualmente.

Esse processo manual tem dois problemas graves: é demorado e cansativo, e — mais importante — depende da atenção e da disposição de quem está executando. A mesma pessoa testa a primeira página com atenção total e a quadragésima no automático, e pode simplesmente não perceber que um botão parou de funcionar.

O objetivo deste projeto é eliminar o fator humano dessa verificação técnica, transformando-a num sistema determinístico e reproduzível: a mesma checagem, executada do mesmo jeito, toda vez — sem cansaço e sem variação.

Escopo restrito e proposital: este sistema é PURAMENTE TÉCNICO. Ele verifica se existe alguma variável técnica influenciando o resultado da loja. Não avalia copy, oferta, design, layout ou qualquer aspecto qualitativo/subjetivo — isso será tratado por um fluxo completamente separado e não faz parte deste projeto.

2. O problema (o que queremos resolver)

- O teste técnico manual da loja é lento e ninguém gosta de fazer, então acaba não sendo feito com consistência.
- A detecção de falhas depende de quem executa: atenção e disposição variam de pessoa para pessoa e ao longo da tarefa, e uma falha técnica pode passar despercebida.
- Não existe um registro histórico padronizado que permita comparar o estado técnico da loja hoje com o de ontem ou da semana passada.
- Quando o faturamento cai, perdemos tempo sem conseguir afirmar com segurança se a parte técnica está ou não influenciando o resultado.

3. Resultado esperado (o desejo)

Queremos um sistema que faça uma verificação técnica completa e padronizada da loja sob demanda (e idealmente de forma agendada/automática), produzindo um relatório objetivo que responda, sem nenhuma subjetividade, à pergunta: "existe alguma variável técnica quebrada ou degradada na loja agora?".

O sistema ideal deve:

1. Percorrer as páginas e fluxos críticos da loja sozinho (home, coleções, páginas de produto, carrinho, checkout) — em desktop e mobile.
2. Verificar o funcionamento real: páginas carregam (status HTTP), botões e elementos interativos respondem, popups disparam e fecham corretamente sem travar o fluxo, o fluxo de compra completa até o final, formulários submetem, links não estão quebrados.
3. Medir saúde técnica objetiva: erros de console/JavaScript, requisições que falham, recursos que não carregam (imagens, scripts, CSS), tempos de carregamento e métricas de performance.
4. Gerar um relatório estruturado com resultado por página e por checagem, sinalizando objetivamente o que passou e o que falhou (passa/falha, com o detalhe técnico do erro).
5. Guardar histórico, para que possamos comparar a saúde técnica da loja ao longo do tempo e ver exatamente o que quebrou ou degradou quando algo muda.

Princípio central: todas as checagens são determinísticas e binárias (ou métricas numéricas). Não há julgamento, opinião ou interpretação em nenhum ponto. Dois testes da mesma página, com a loja no mesmo estado, devem dar exatamente o mesmo resultado.

4. Como pensar a solução (sem fechar a arquitetura)

A divisão abaixo é uma sugestão conceitual para orientar o raciocínio — não uma especificação fechada. Você tem liberdade para explorar e propor as melhores ferramentas e abordagens. Todo o sistema é determinístico: não há nenhuma camada de IA, julgamento ou avaliação subjetiva.

Camada A — Verificação funcional dos fluxos

Automação de navegador percorrendo os caminhos reais da loja: home → coleção → página de produto → adicionar ao carrinho → checkout, em desktop e mobile. Cada passo é uma asserção binária: o elemento existe? o clique funcionou? a próxima etapa carregou? o fluxo completou? Qualquer passo que falhe é registrado com o detalhe técnico. Vale pesquisar ferramentas de automação de navegador (ex.: a categoria de Playwright/Puppeteer).

Os popups da loja entram nesta camada e merecem atenção específica, porque quebram de forma silenciosa e podem travar a compra inteira. Testar, para cada popup (newsletter, cupom, intenção de saída, cookies, avisos, etc.): se ele dispara no gatilho correto, se aparece nos dois dispositivos (desktop e mobile), se o botão de fechar funciona, se ele não bloqueia o scroll nem o clique nos elementos por baixo, se não impede o fluxo de compra, e se não dispara em loop ou em momento indevido.

Camada B — Saúde técnica das páginas

Para cada URL crítica, coletar sinais técnicos objetivos: status HTTP, erros de console e exceções de JavaScript, requisições de rede que falharam, recursos que não carregaram (imagens, scripts, fontes, CSS), e métricas de performance/carregamento. Tudo isso é numérico ou binário — sem interpretação.

Camada C — Relatório e histórico

Consolidar os resultados das camadas A e B num relatório legível e num histórico comparável ao longo do tempo. O formato fica a seu critério (relatório, planilha, dashboard) — o importante é que seja fácil de ler, que aponte exatamente qual checagem falhou e em que página, e que permita comparação temporal.

5. Entregáveis sugeridos

- Um projeto versionado no GitHub, construído com Claude Code.
- Capacidade de rodar a auditoria sob demanda (manualmente) e de forma agendada.
- Um arquivo de configuração versionado e editável — separado do código — com a lista de URLs, fluxos e checagens a executar, para ajustarmos o que é monitorado sem mexer na lógica.
- Relatório de saída por execução + histórico acumulado.
- Um README explicando como rodar, como adicionar/editar checagens e como adicionar/remover URLs e fluxos monitorados.

6. Restrições e princípios inegociáveis

6. Determinismo total: mesmo input gera exatamente o mesmo resultado. Não há nenhuma fonte de subjetividade ou variação no sistema.
7. Escopo puramente técnico: o sistema só verifica funcionamento e saúde técnica. Nada de copy, oferta, design ou qualquer avaliação qualitativa.
8. Saída estruturada e limpa: passa/falha e métricas numéricas, com o detalhe técnico do erro quando há falha. Sem prosa, sem interpretação, campos nulos preservados.
9. Configurável sem mexer no código: lista de páginas, fluxos e checagens devem ser editáveis por quem não programa.
10. Baixo custo de operação: a verificação vai rodar com recorrência, então atenção a manter a execução enxuta e rápida.

7. Fora de escopo (importante alinhar)

Este sistema verifica exclusivamente a saúde técnica da loja. Ele NÃO avalia copy, oferta, headline, design, layout, prova social ou qualquer aspecto qualitativo/subjetivo — isso será tratado por um fluxo totalmente separado, fora deste projeto. Ele também NÃO é um diagnóstico da causa da queda de faturamento, que pode estar em CPM, criativo, sazonalidade, tracking/atribuição ou entrega da conta de anúncio.

O valor real do sistema é descartar (ou confirmar) a hipótese técnica com rapidez e sem viés, garantindo que nenhuma variável técnica esteja influenciando o resultado. Isso libera o time para investigar as demais hipóteses (incluindo copy/oferta, no fluxo próprio) sabendo que a parte técnica já foi isolada.

8. Decisões deixadas para você (dev)

Queremos sua iniciativa nestes pontos — explore, compare opções e proponha o melhor:

- Quais ferramentas usar em cada camada (automação de navegador, coleta de erros de console/rede, auditoria de performance).
- Como estruturar o arquivo de configuração de URLs/fluxos/checagens e o formato exato da saída.
- Como definir os limites das métricas (ex.: a partir de quanto um tempo de carregamento conta como falha).
- Formato final do relatório e do histórico (arquivo, planilha, dashboard).
- Estratégia de agendamento e disparo manual.

Antes de começar a codar, monte um plano rápido (estrutura de pastas + abordagem de cada camada + formato da saída) e valide com a gente. Depois disso, mão na massa.

9. Quadro-resumo

Item	Descrição
Problema	Teste técnico manual da loja é lento e uma falha pode passar despercebida dependendo de quem executa.
Desejo	Sistema determinístico que responde "existe alguma variável técnica quebrada/degradada na loja agora?".
Como	Camada funcional (fluxos) + camada de saúde técnica (erros, rede, performance) + relatório/histórico. Tudo determinístico, sem IA.
Inegociável	Determinismo total; escopo puramente técnico (sem copy/oferta/design); saída estruturada; configurável sem código.
Fora de escopo	Não avalia copy/oferta/design (fluxo separado) e não diagnostica a causa da queda — apenas isola a hipótese técnica.
Stack	GitHub + Claude Code; ferramentas de cada camada à escolha do dev.

Qualquer dúvida de escopo, alinhar antes de codar. Liberdade total na execução técnica.